

Tour Of The Demo

[Previous: Getting Started In 10 Minutes](#) | [Documentation Home](#) | [Next: Overview](#)

The Spring Boot demo is not just a smoke test. It is a small application that shows how Coredeux is meant to be used inside a real enterprise-style host.

After the quick start, this is the chapter that helps you understand what the demo is teaching.

If you are a developer, this page shows where the demo code lives and what each part is responsible for. If you are an AI agent reading the repository, this is the map of the application surface you are expected to work with.

If you want to compare it with the plain Java version of the demo, see:

- [Native Getting Started In 10 Minutes](#)
- [Native Tour Of The Demo](#)

The Point Of The Demo

The demo exists to show that the framework can stay consistent while the application grows different kinds of behavior around it:

- PostgreSQL-backed CRUD
- JDBC, MongoDB, Elasticsearch, and Redis examples
- validation
- hooks
- audit
- import and file import
- export
- custom workflow behavior

The key idea is that the demo does not reinvent the framework for each feature. It keeps one Coredeux shape and adds application-specific behavior around it.

Where The Demo Lives

The demo application is under:

```
examples/coredeux-spring-boot-demo
```

The important parts are:

- `src/main/resources/application.yml`
 - application defaults
 - database connection settings
 - Coredeux property defaults

- export settings
- Swagger/OpenAPI setup
- `src/main/resources/coredeux-entities.yml`
 - managed entity definitions
 - entity identifiers
 - storage adapters
 - enabled modules
- `src/main/java/com/coredeux/demo/domain`
 - the main JPA model
- `src/main/java/com/coredeux/demo/domain/jdbc`
 - a JDBC-backed example entity
- `src/main/java/com/coredeux/demo/domain/mongodb`
 - a MongoDB document example
- `src/main/java/com/coredeux/demo/domain/elasticsearch`
 - an Elasticsearch document example
- `src/main/java/com/coredeux/demo/domain/redis`
 - a Redis-backed example entity
- `src/main/java/com/coredeux/demo/web`
 - generic CRUD, import, file import, and export controllers
- `src/main/java/com/coredeux/demo/validation`
 - validation examples
- `src/main/java/com/coredeux/demo/hooks`
 - lifecycle hook examples
- `src/main/java/com/coredeux/demo/audit`
 - audit handler example
- `src/main/java/com/coredeux/demo/imports`
 - custom import value handlers
- `src/main/java/com/coredeux/demo/export`
 - custom export value handlers and storage behavior
- `src/main/java/com/coredeux/demo/workflow`
 - custom module example
- `src/main/java/com/coredeux/demo/bootstrap`
 - startup data and search examples
- `samples`
 - import files used by the file-import flow

The Story In The Demo

The easiest way to understand the demo is to read it as a series of layers.

1. The entity definition layer

The demo starts with `coredeux-entities.yml`.

That file tells Coredeux:

- which entity exists
- which field is the identifier
- which storage implementation should handle it
- which modules should run for it

This is the first important pattern in Coredeux: behavior starts from the entity definition, not from a pile of special-case controllers.

2. The controller layer

The generic CRUD controller shows how one endpoint shape can work across multiple entity types.

The demo does not need a different controller for each entity. The path tells Coredeux what to load, and the framework routes the work through the correct backend.

That is the second important pattern: the application stays small because the framework keeps the behavior consistent.

3. The module layer

The demo shows the framework modules working around the entity lifecycle:

- validators for business rules
- hooks for lifecycle behavior
- audit handlers for operation tracking
- workflow-style custom modules

This is where Coredeux becomes more than persistence. The entity is still the center, but the framework can surround it with business behavior in a way that stays organized.

4. The import and export layer

The demo also shows how Coredeux handles import and export as first-class application flows.

Import is not treated as a one-off parser tucked into a controller. Export is not treated as a random file download either. Both are framework-driven flows with application-specific extensions on top.

5. The startup layer

`DemoDataRunner` and the sample import files show how the app can seed itself and then keep working with the same Coredeux flow.

That makes the demo easier to explore and easier to teach.

What To Notice First

When you read the demo, start here:

1. `coredeux-entities.yml`
2. the generic CRUD controller
3. the validator, hook, and audit handlers
4. the workflow module
5. the import handlers and import controllers
6. the export handler and export storage service
7. `DemoDataRunner`
8. the sample import files

That order shows the framework from core contract to application behavior.

Why The Demo Matters

The demo is important because it proves a practical point:

Coredeux is not only a framework for abstract design. It is a way to build a real application where the same conventions keep working across multiple backends and multiple kinds of business behavior.

That is what makes it useful for both developers and future agent-driven workflow tooling.

What To Copy Into Your App

You usually do not copy the demo as-is. You copy the patterns:

- entity definitions in YAML
- controllers that delegate to Coredeux
- validator beans
- hooks
- audit handlers
- custom modules
- import value handlers
- export value handlers
- export storage services

Those are the reusable ideas the demo is showing.

What To Read Next

If you want the deeper architecture view, continue with:

- [Overview](#)
- [Coredeux Core Reference](#)
- [Lifecycle Model](#)

[Previous: Getting Started In 10 Minutes](#) | [Documentation Home](#) | [Next: Overview](#)